Assignment 1

Due date: 23:59 2026-04-20

The purpose of this assignment is to gain good understanding of the ordering property of multicast primitives through implementing the total order multicast primitive.
Many applications need to use multicast mechanisms that satisfy various ordering properties to send information. To simplify the task of application development, many systems provide multicast mechanisms using middleware or OS services. In this assignment, you are required to implement a middleware that ensures total order of multicast messages.

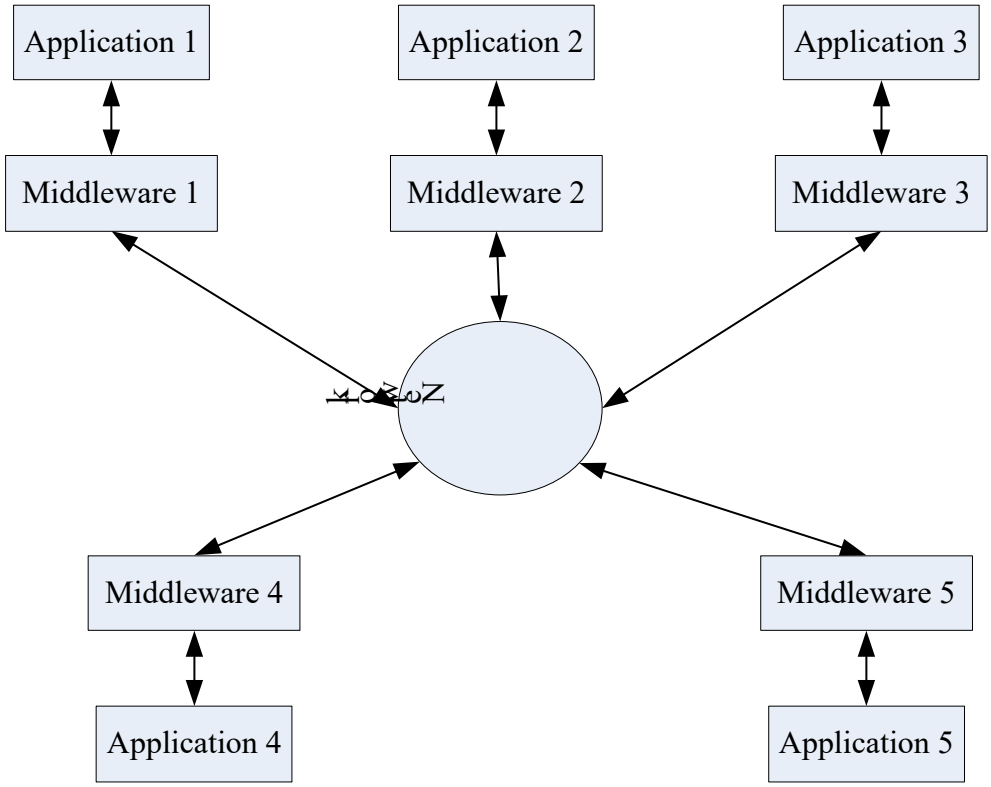


Figure 1 Multicasting through Middleware

Figure 1 shows how the middleware is used.

- Each application program has a corresponding middleware. An application and its corresponding middleware reside on the same machine.
- The middleware uses a protocol to guarantee the ordering property of the multicast messages.
- When an application needs to multicast a message to other applications, the application sends the message to its corresponding middleware. The middleware is responsible for multicasting the message to the corresponding middleware of the other applications through the network.
- When a middleware receives a multicast message from the network, it determines whether the message can be delivered to its corresponding application according to

the protocol for guaranteeing the message ordering. The middleware buffers a message until the message can be delivered to the application.

In this assignment, you are provided with a program for simulating the network. The multicast messages sent by the middleware on behalf of the application programs MUST be sent to the network program first. Subsequently, the network program will multicast the message to all the middleware in the system.
Except for modifying the values stored in the file 'delays.txt', you MUST not alter any other file in the 'network' folder. The marker will employ the same network program during the assessment of your assignment. However, the marker may utilize a 'delays.txt' file with a different set of values. An example, which includes the network program and some simple programs for testing the network program, can be downloaded from Canvas.
The details of the network program are as below:

- The middleware communicates with the network program through TCP sockets.
- The network program receives multicast messages sent by the middleware and broadcasts each received message to ALL the middleware in the system. The broadcast is executed via point-to-point communication using TCP sockets.
- The network program listens for incoming messages on port 8081.
- File 'delays.txt' comprises several lines, with each line containing several values separated by ','. The number of values in each line is equivalent to the number of middleware in the system. These values represent the amount of delays that the network program uses when sending a multicast message to middleware. Each value in a line corresponds to a middleware. For example, if there are two middleware M1 and M2, then each line of 'delays.txt' consists of two values, i.e., d1 and d2, and they are the amount of delay for M1 and M2 respectively.
- To simulate network delay, while broadcasting a message, the network program waits for a specific duration before sending the message to the middleware. The waiting time for different middleware may vary and is determined by the values in the file 'delays.txt'. For instance, assuming there are two middleware, M1 and M2, and the contents of 'delays.txt' are as follows:
  100,2000
  1000,30
  …
  When the network receives the first multicast message (msg1), it reads the two values in the first line of 'delays.txt'. Based on these values, the network program will wait for 100ms and 2000ms before sending msg1 to M1 and M2, respectively.
  Similarly, when the network receives the second multicast message (msg2), it reads the two values in the second line of 'delays.txt'. Accordingly, the network program will wait for 1000ms and 30ms before sending msg2 to M1 and M2, respectively.
- Port 8081 is utilized by the network program for receiving multicast messages from the middleware.

In this assignment, you are required to implement programs corresponding to Middleware 1 to 5 in Figure 1. You do not need to provide any of the application programs shown in Figure 1. You must use C# to do this assignment.

Programming (12 marks)

In this part, you are required to implement the middleware that ensures the total order of the multicast messages. In order to test the middleware, your implementation must include a GUI interface for the middleware. The layout of the interface for each middleware should be similar to Figure 2.

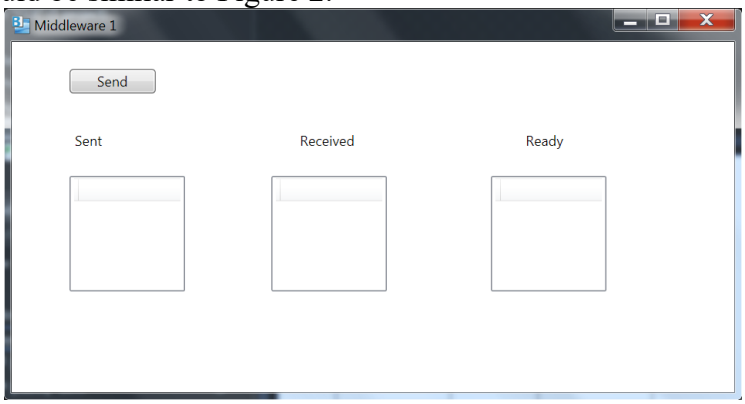


Figure 2 The GUI for a Middleware

The middleware and its GUI should satisfy the requirements below:

- Each middleware should have its own GUI.
- A GUI should have a title indicating the ID of the middleware, e.g. "Middleware 1".
- Each GUI should have a button "Send". When the button is clicked, the middleware should send a multicast message to the network. The contents of the message should be generated automatically by your program. Each multicast message sent by a middleware must include information that uniquely identifies the message. For example, "Msg #1 from Middleware 1 xxx" indicates that this is the first message sent by Middleware 1[1]. A message must be less than 1024 bytes.
- Each GUI should have three lists for displaying messages.
    - The "Sent" list shows the multicast messages that have been sent by the middleware to the network. The messages in the list should appear in the order in which they were sent to the network.
    - The "Received" list contains the multicast messages that have been received by the middleware from the network. The received messages should be displayed in the order in which they are received by the middleware. You should also show the proposed timestamp assigned to each message.
    - The "Ready" list presents the multicast messages that are ready to be delivered to the application program. This means that the messages in the "Ready" list should be a subset of the messages in the "Received" list. The order in which the messages appear in the "Ready" list should conform to the total-order property of the multicast messages. You should also show the final timestamp assigned to each message.
- Your implementation should have five middleware programs. Each middleware program should be stored in its own folder. There is NO need to run your programs concurrently on different machines. You only need to test them on one machine.

[1] #1 means the first message. In this case, "Msg #1 from Middleware 1" uniquely identifies the message. "xxx" denotes other information in the message, e.g. timestamps.

- Each middleware program should create a TCP socket for receiving messages from the network program. The sockets used by the five middleware programs MUST use ports 8082, 8083, 8084, 8085, and 8086 respectively.
- The middleware programs might need to exchange some "control" information amongst them when determining the order in which the multicast messages should be delivered. The middleware programs must exchange the control information between them directly using TCP sockets. That is, the control messages should not be sent to the 'Network'.
    - On lab machines, you can only use port 8080 to 8099. Please make sure that your implementations take this into account. Your implementations MUST be able to run on a lab machine.
- You must NOT use any absolute path in your code, e.g., 'c:xyz\A1'. Given that your directory structure may differ from the marker's, using absolute paths may cause your program to fail to execute correctly on the marker's machine. Failure to execute correctly will result in receiving a mark of 0.

Report (3 marks)

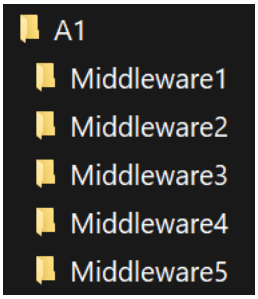
Write a report about your implementation. Your report should include the following.

(a) Clearly indicate whether you have tested your programs on a lab machine.
(b) Instructions on how to run your programs.
(c) What information is used to uniquely identify a multicast message.
(d) Describe an application scenario in which the total-order multicast is needed to ensure the consistency of the data in the system. You must provide sufficient explanations as why total-order multicast is needed in the scenario.
(e) Use an example to show the difference between total order and causal order multicast.

Save your report as a PDF file. Name it as *Report.pdf*.

Submission

- You MUST thoroughly test your programs on a lab machine. Programs that cannot be compiled or run on a lab machine will get **NO** marks.
- You only need to submit the implementations of the middleware. Your implementations should have the following directory hierarchy. The directory structure must be similar to the example program.



- You should only submit the source code of your programs.

- You should create a windows batch file for compiling the submitted programs and for executing the programs. Name the batch file as run.bat. The batch file should be placed under folder ‘A1’.
- Pack your implementations and the batch file into file A1.7z. The directory hierarchy of your implementation should be preserved when you pack your files.
- Submit **A1.7z** and **report.pdf** to Canvas.
- **NO** email submission will be accepted.

Marking Schedule

Programming

1. The batch file for compiling and executing programs works correctly. [0.5 marks]
2. There are five middleware programs and the programs are placed in the right folders. [0.5 marks]
3. Each GUI shows the ID of the middleware. [0.5 marks]
4. When the "Send" button is clicked, a multicast message is sent to the network program. [0.75 marks]
5. Each multicast message contains the information that uniquely identifies the message. [0.75 marks]
6. Multicast messages are added to the "Sent" list correctly. [2 marks]
7. Multicast messages are added to the "Received" list correctly. [2 marks]
8. Multicast messages are added to the "Ready" list correctly. [5 marks]

Report

1. Statement about testing on a lab machine. [0.1 mark]
2. Clear instructions on running the programs. [0.5 marks]
3. Information on how to uniquely identify a multicast message. [0. 5 marks]
4. Clear description about an application scenario. [1 mark]
5. Clear demonstration of the difference between total order and causal order. [0.9 marks]